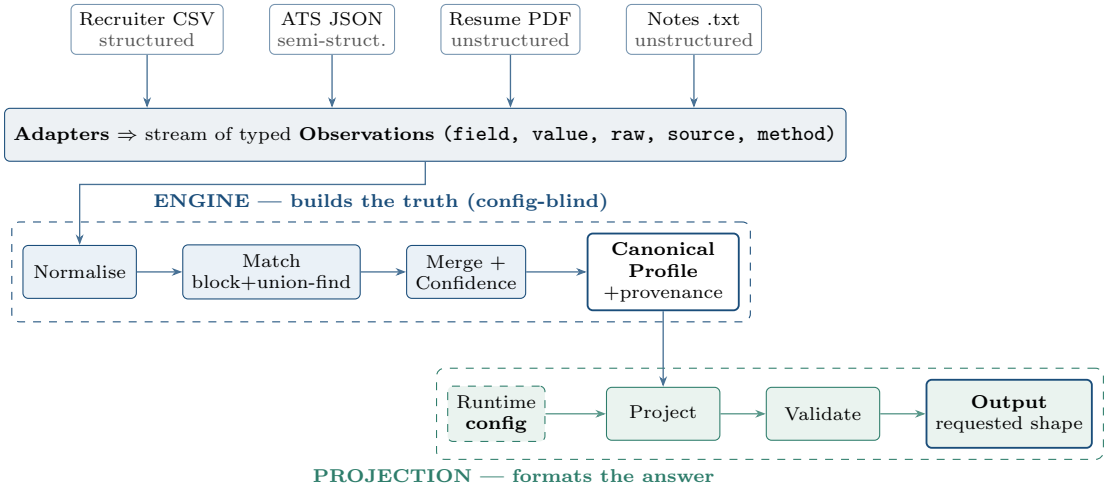
Multi-Source Candidate Data Transformer

Stage 1 — Technical Design • **Harsh** • harshsahuhh@gmail.com

North star: the system optimises for one thing — *never be confidently wrong.* "Wrong-but-confident is worse than honestly-empty." Every design choice below is derived from that principle plus the brief's four constraints: **deterministic, explainable, robust, scalable**.

Architecture — truth is built once, then projected



1 Pipeline

`detect` → `extract` → `normalise` → `match` → `merge` → `project` → `validate`. Each adapter turns one raw source into observations; nothing downstream sees a raw source again, so **adding a source = one adapter, zero changes elsewhere**. The engine builds one fixed internal record; projection is the only config-aware code. *Why:* this boundary is what keeps output changes from ever touching resolution logic.

2 Canonical schema & normalised formats

Field	Normalised form
phones	E.164 (`+14155550182`)
location	ISO-3166 alpha-2 country
links	object {`linkedin, github, portfolio, other[]`}
experience	dates → `YYYY-MM` (+ `present`), + summary
education	{`institution, degree, field, end_year`}
skills	canonical names (`JS`→JavaScript)
emails	lower-cased, deduped, typo-suppressed

A normaliser that cannot parse a value returns `null`; the value is dropped, never invented.

3 Match, merge & confidence

Match keys (priority): email → phone → name+company; matches clustered by *blocking + union-find* so cost stays near-linear at thousands of records. **Each observation has**

$$s \;=\; \text{trust}_{\text{source}} \times \text{reliability}_{\text{method}}$$

The strongest camp wins a single-valued field; multi-valued fields union+dedupe. **Confidence** is a noisy-OR over agreeing evidence

$$c \;=\; 1 - \prod_i (1 - s_i)$$

so corroboration *raises* confidence (an average wouldn't) and conflict *discounts* the winner. Below a floor, a single value is emitted `null`. Trust weights live in a config table (policy), separate from merge logic (mechanism).

4 Configurable output (the twist)

Projection selects a subset, remaps via a `from`-path language (`emails[0]`, `links.github`, `skills[].name`), toggles confidence/provenance, and applies a per-field policy `null | omit | error`, then validates against the *requested* schema.

```
{ "fields": [
  {"path":"full_name","required":true},
  {"path":"primary_email","from":"emails[0]"},
  {"path":"phone","from":"phones[0]","normalize":"E164"},
  {"path":"github","from":"links.github","on_missing":"
      omit"}
], "include_confidence": true }
```

⇒ `{"full_name":"Maya R. Chen", "primary_email":"maya.chen@example.com", "phone":"+14155550182"}`
(`github` omitted — no source had it).

5 Edge cases

Case	Handling
phone in 3 formats	all → one E.164; corroborated
typo'd email	near-dup of verified → suppressed
title conflict	higher-trust source wins; logged
`Java` vs `JavaScript`	fuzzy match refused below 88%
corrupt source	yields 0 obs, logs, run continues
field nobody has	stays `null`; never invented

6 Rejected alternatives & descoping

LLM/ML resolution — rejected: non-deterministic and untraceable, and would confidently mis-map `Java`→`JavaScript` (the exact failure we must avoid). **Last-write-wins merge** — rejected: can't explain or score. **Deliberately out of scope under time:** a UI (CLI suffices per brief), a datastore, and ML entity-resolution — blocking + strong keys is the right complexity here.